Rashtreeya Sikshana Samithi Trust

R. V. COLLEGE OF ENGINEERING

[Autonomous Institution Affiliated to VTU, Belagavi]

Department of Computer Science & Engineering

Bengaluru-560059

Design and Analysis of Algorithms

Subject Code: CD343AI

Sudoku Solver: Backtracking Algorithm Implementation

Submitted By

Akshat Arya **1RV23CS026**

Amol Vyas **1RV23CS032**

Under the Guidance of

Dr. Krishnappa H K

IV SEMESTER B.E.
Computer Science Engineering
EL Lab Report

[Autonomous Scheme 2022]

2024-2025

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled ***'Sudoku Solver: Backtracking Algorithm Implementation'*** is carried out by **Akshat Arya (1RV23CS026)** and **Amol Vyas (1RV23CS032)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering** of the **Visvesvaraya Technological University**, Belagavi during the academic year 2024-2025. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Guide
Dr. Krishnappa H K

Signature of Head of the Department
Dr. Shanta Rangaswamy

External Viva

Name of Examiners	**Signature with Date**
1	
2	

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We**, Akshat Arya** and **Amol Vyas,** students of fourth semester B.E., department of CSE, RV College of Engineering, Bengaluru, hereby declare that Experiential Learning (Lab) titled ***'Sudoku Solver: Backtracking Algorithm Implementation'*** has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Computer Science and Engineering** during the academic year 2024-25

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature
1. **Akshat Arya (1RV23CS026)**	
2. **Amol Vyas (1RV23CS032)**	

Abstract

Sudoku, a popular logic-based puzzle, challenges players to fill a 9x9 grid with digits from 1 to 9, adhering to specific constraints. This project presents the development of a Sudoku solver using C++, demonstrating effective algorithm design and implementation. The program leverages backtracking, a widely-used algorithmic approach for constraint satisfaction problems, to efficiently explore potential solutions. In addition to the core solver, the project includes a graphical user interface (GUI) for user interaction, enhancing the overall user experience.

The solver's backtracking algorithm systematically fills the Sudoku grid, recursively checking for valid placements of numbers. The approach begins by selecting an empty cell and testing possible digits, backtracking when a conflict arises. This method ensures that all potential solutions are evaluated while minimizing redundant checks through intelligent heuristics, such as selecting the most constrained variable first. The efficiency of the algorithm is paramount, particularly for complex puzzles requiring numerous iterations.

To manage user interactions and provide a visual representation of the Sudoku puzzle, the project employs a GUI, developed with a focus on usability. The main window allows users to input initial values effortlessly, while visual feedback indicates valid placements and conflicts. This graphical layout not only aids in the debugging process but also enhances the educational component of the project, making it accessible for individuals seeking to learn more about Sudoku strategies and algorithm design.

In conclusion, this project integrates algorithmic theory with practical application, resulting in a functioning Sudoku solver that is both effective and user-friendly. By utilizing C++ for implementation, the project showcases the capabilities of programming in terms of performance and user interface development. Future work could explore the integration of advanced solving techniques, such as constraint propagation and machine learning, to further improve the solver’s efficiency and broaden its range of applicability in more complex puzzle variations.

Introduction

Sudoku, a widely recognized logic-based puzzle, has captivated enthusiasts across the globe for decades. The objective is straightforward: fill a 9x9 grid with digits from 1 to 9, ensuring that each row, column, and 3x3 sub-grid contains all digits without repetition. While the rules are simple, the challenge lies in solving it efficiently, especially for complex puzzles with minimal clues. This project addresses the inherent difficulty of the Sudoku problem by implementing a robust solver that automates the solution process through algorithmic techniques, significantly reducing the time and effort involved for users.

The significance of solving Sudoku puzzles goes beyond entertainment; it serves as an excellent case study in algorithm design and analysis. The constraints and logical reasoning required to solve Sudoku puzzles make it an ideal candidate for exploring various algorithmic strategies, including backtracking, constraint satisfaction, and heuristics. By developing a Sudoku solver, this project not only contributes to the understanding of these concepts but also provides insights into the performance of different algorithms. Furthermore, the ability to solve Sudoku puzzles quickly and efficiently can promote interest in computational thinking and problem-solving skills, as well as foster a greater appreciation for algorithms in computer science.

The primary objective of this project is to create an efficient Sudoku solver using backtracking, one of the simplest yet effective algorithms for constraint satisfaction problems. The project aims to develop a clear and concise implementation in C++, enabling the solver to navigate through potential solutions systematically. In addition, a graphical user interface (GUI) will be integrated, allowing users to interact with the program intuitively. This GUI will facilitate ease of input for initial puzzle states and provide a visual representation of the solving process, enhancing user engagement and educational value.

Ultimately, the proposed solution encapsulates an intersection of logic, programming, and user experience, showcasing the practicality of algorithms in real-world applications. By leveraging algorithmic principles, the Sudoku solver not only aims to provide accurate answers but also serves as a valuable learning tool for students and puzzle enthusiasts alike. The successful implementation of this project will demonstrate the power of computational techniques in transforming complex problems into manageable solutions.

Solution Design

The proposed solution for the Sudoku solver is designed to utilize a backtracking algorithm, which is a form of depth-first search (DFS) specifically tailored for constraint satisfaction problems. The system is structured to include the core solver logic, user input handling, and a graphical user interface (GUI) for ease of interaction. The overall architecture consists of several components:

- **Sudoku Grid Representation**: The Sudoku puzzle will be represented as a 2D array (or matrix) of integers, where each cell can either hold a number between 1 and 9 or be empty (indicating unfilled). This representation allows for straightforward access and modification of individual cell values.
- **Backtracking Algorithm**: The core logic of the Sudoku solver will implement a backtracking algorithm that recursively attempts to fill the grid. The algorithm will start from the first empty cell, attempting to place each number from 1 to 9, using the validator class to check for conflicts. If a number placement leads to a valid state, the algorithm proceeds recursively to the next empty cell. If a conflict arises, it backtracks and tries the next number.
- **Graphical User Interface (GUI)**: The GUI will be developed using a framework such as Qt or similar, allowing users to interact with the solver intuitively. The interface will include input fields for the initial Sudoku configuration, buttons to start the solving process, and visual cues to indicate the current state of the puzzle during execution.

Selection of Data Structure:

- **2D Vector (SudokuGrid):**
  - A std::vector<std::vector<int>> is used to represent the 9x9 Sudoku grid. This dynamic structure is flexible, allowing for easy resizing and manipulation typical in grid-based applications.
  - Benefits of using a vector over a static array include automatic memory management and easier integration with algorithms that require grid traversal.
- **Qt Widgets:**
  - QLineEdit widgets are used for each cell in the Sudoku grid, allowing user input and displaying the numbers. This suits the needs for read-only states to show original numbers and solved numbers distinctly.

Algorithmic Strategies:

- **Backtracking Algorithm**:
  - The primary strategy employed for solving Sudoku is backtracking, a depth-first search algorithm where numbers are tentatively placed in cells, and the algorithm recursively attempts to fill the grid. If a number leads to an unsolvable condition, the algorithm backtracks to try the next

number.
    - This strategy is justified since it explores all possible configurations and guarantees a solution for valid puzzles within polynomial time.
- **Random Generation and Validation**:
    - When creating puzzles, the algorithm first generates a completely solved Sudoku grid, then randomly removes numbers while ensuring a unique solution using recursive checking, which is efficient for generating valid puzzles.

Time Complexity Analysis:

1. **Time Complexity**:
    - The worst-case time complexity for the backtracking algorithm is O(9^n^2), where n is the number of empty cells (in this case, n=9). However, in practical scenarios, this is significantly reduced through early exit conditions and validating placements, leading to much quicker solving times.
2. **Space Complexity**:
    - The space complexity primarily results from the recursive stack for backtracking, which can go up to O(n^2) for the grid size. However, since only a constant amount of extra space is required for each cell placement, it remains manageable.

Implementation Details

The Sudoku solver project represents a comprehensive approach to solving complex puzzle problems through an intelligently designed software architecture. At its core, the implementation leverages a recursive backtracking algorithm, which serves as the primary problem-solving mechanism. The project is structured around two primary classes: SudokuSolver, which encapsulates the core algorithmic logic, and MainWindow, which manages user interactions and visualization.

The software design emphasizes strict modularity and separation of concerns. The SudokuSolver class is meticulously crafted to handle distinct responsibilities, including puzzle validation, solving, and generation. By utilizing static methods, the implementation ensures efficient memory management and prevents unnecessary object instantiation. Each method is designed with a single, well-defined purpose - for instance, isSafe() systematically checks number placement validity across rows, columns, and 3x3 sub-grids, while solveSudoku() manages the recursive solving process.

Readability and maintainability were paramount in the code's design. The implementation adopts consistent naming conventions, using descriptive camelCase method names that clearly communicate their functionality. Type definitions, such as using SudokuGrid = std::vector<std::vector<int>>, provide flexibility in data structure representation while maintaining clean, abstracted code. The use of modern C++ features and standard library components demonstrates a commitment to writing idiomatic, efficient code that is both easy to understand and modify.

The heart of the Sudoku solver is its recursive backtracking algorithm, which employs a depth-first search strategy to solve puzzles. The solveSudoku() method recursively explores possible number placements, systematically backtracking when an invalid configuration is detected. Performance optimizations are seamlessly integrated into this approach. The isSafe() method acts as an early constraint satisfaction mechanism, dramatically reducing unnecessary recursive calls by preemptively validating number placements.

The Sudoku generation and solving process leverages a sophisticated backtracking algorithm with constraint satisfaction techniques. Initially, the generator constructs a complete valid grid through a recursive depth-first search strategy, systematically filling cells while maintaining the core Sudoku constraints: unique numbers in rows, columns, and 3x3 sub-grids. The solving algorithm employs a similar recursive backtracking mechanism, utilizing a recursive function that explores potential number placements through systematic trials and eliminations. When solving, the algorithm uses a findEmptyCell() method to locate unassigned grid positions, then attempts to place numbers 1-9 while continuously validating against grid constraints through an isSafe() validation function. If a number placement violates Sudoku rules, the algorithm automatically backtracks, effectively pruning invalid solution paths.

For puzzle generation, the solver strategically removes numbers from a fully solved grid while ensuring puzzle uniqueness, maintaining a delicate balance between randomness and solvability through probabilistic number removal techniques and solution verification algorithms.

Results and Discussion

The Sudoku solver successfully demonstrated its effectiveness by accurately solving a wide variety of Sudoku puzzles while adhering to the core rules of the game. During testing, the implementation consistently verified that all solutions satisfied the fundamental constraints, ensuring that each number from 1 to 9 appeared only once in every row, column, and 3x3 sub-grid. This correctness was validated through rigorous checks applied during the solving process.

The algorithm efficiently handled multiple special and edge cases, including scenarios where puzzles had minimal initial numbers or required unique solutions. By incorporating explicit mechanisms for recognizing unsolvable configurations, the solver was able to prevent infinite recursion by backtracking as soon as a violation of Sudoku rules was detected. This capability ensured that the solver remained robust and efficient, even when encountering challenging scenarios.

The complexity analysis of the algorithm revealed important insights into its performance characteristics. The worst-case time complexity of the backtracking algorithm is theoretically O(9^n^2), where n represents the dimensions of the grid (in this case, n=9). However, in practice, the solver's performance is significantly better due to effective constraint satisfaction techniques. The use of backtracking allows the algorithm to quickly eliminate invalid placements and focus on viable paths, leading to improved average-case performance.

The space complexity of the implementation is $O(n$^2) since it requires storage for the 9x9 grid as well as auxiliary data during the recursive solving process. The recursive call stack contributes to this space usage, although it is managed effectively through careful algorithm design.

In comparison to alternative algorithms, such as brute force techniques and constraint propagation methods, the backtracking approach used in this project strikes a favorable balance between simplicity and efficiency. While brute force methods can be straightforward to implement, they tend to be less efficient due to their exhaustive nature. On the other hand, constraint propagation methods require more complex implementations but can significantly reduce search space. The current backtracking algorithm minimizes unnecessary computations through strategic pruning, making it an ideal choice for a Sudoku solver.